Version corrigée du 2026-02-11

TD 1

Théorie des jeux

Exercice 1 : Stratégies au jeu de Nim . . . . . [⋆]

Nous étudions dans tout cet exercice le jeu de Nim ($n = 5, p = 2$).

Implémentation du jeu — On commence par implémenter le graphe du jeu sous forme de dictionnaire.

1) Représenter graphiquement un graphe orienté *simple* G_s associé au jeu de Nim ($n = 5, p = 2$). Définir formellement le graphe $G_s = (S, A)$, ainsi que l'origine s_0 et les sommets finaux F.

Solution

Voir cours. Le graphe orienté $G_s = (S, A)$ est la donnée d'un ensemble de sommets S et d'arcs A.

(a) *En nommant le sommet i par le nombre d'objets restants, on a au plus simple,*

$$\boxed{S = \{5, 4, 3, 2, 1, 0\},}$$

qui comprend $m = 6$ éléments.

(b) *L'ensemble des arcs A est donné par*

$$\boxed{A = \{(4, 3), (4, 2), (3, 2), (3, 1), (2, 1), (2, 0), (1, 0)\}.}$$

(c) *Le sommet initial (origine) est*

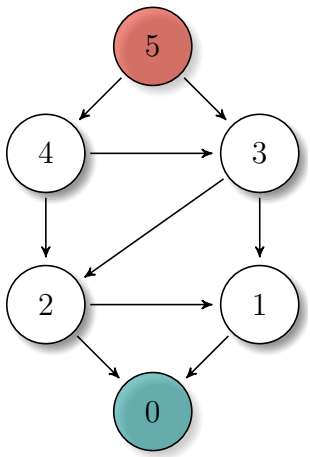
$$\boxed{s_0 = 5.}$$

(d) *L'ensemble des sommets finaux est*

$$\boxed{F = \{0\}.}$$

Solution

Le graphe peut se représenter de la manière suivante.



2) Rappeler les définitions d'une matrice d'adjacence et d'une liste d'adjacence. Donner la matrice d'adjacence et la liste d'adjacence du graphe G_s défini précédemment. Quelle est la complexité $C(n, p)$ pour implémenter la liste d'adjacence du graphe simple d'un jeu de Nim (n, p) ?

Solution

Matrice d'adjacence :

matrice carrée $M \in \mathscr{M}_m(\{0,1\})$, où $m = card(S)$ est l'ordre du graphe, dont les entrées vérifient

$$\boxed{M_{ij} = \begin{cases} 1 & si\ (i,j) \in \mathscr{A} \\ 0 & sinon. \end{cases}}$$

Autrement dit, l'élément M_{ij} vaut 1 si le sommet j est un successeur du sommet i, et 0 sinon. La matrice a autant de lignes et de colonnes que de sommets du graphe. Pour le graphe simple G défini précédemment, la matrice d'adjacence est :

$$\boxed{M = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}}.$$

Remarque.

- *La ligne $i = 0$ remplie entièrement de zéro (sans successeurs) correspond aux sommets finaux.*
- *La colonne $j = 5$ remplie entièrement de zéro (sans antécédents) correspond à l'origine.*

Solution

Liste d'adjacence :

liste L de listes $L_i, i \in \{0, \ldots, m-1\}$, vérifiant

$$\boxed{L_i = \{j \in \{0, \ldots, m-1\} : (i,j) \in \mathscr{A}\}.}$$

Autrement dit, L est la liste des listes L_i des successeurs de chaque sommet i. Pour le graphe simple G_s défini précédemment, la liste d'adjacence est :

$$\boxed{L = [\,[\,], [0], [0,1], [1,2], [2,3], [3,4]\,].}$$

Déterminons la complexité $C(n,p)$ pour implémenter la liste d'adjacence du jeu de Nim (n,p).
Il faut définir pour cela :

- *une liste de p successeurs pour chacun des $n+1-p$ sommets non antécédents du sommet final ;*
- *les listes de $1 + \ldots + (p-1) = \frac{p(p-1)}{2}$ successeurs pour les p sommets antécédents du sommet final.*

On a donc

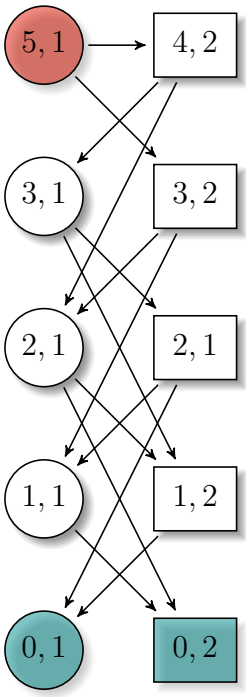
$$\boxed{C(n,p) = p\left(n + 1 - \frac{p+1}{2}\right) = O(mp),}$$

où $m = n+1$ est l'ordre du graphe. On peut comparer à la complexité de la matrice d'adjacence, qui est $O(m^2)$. Pour p petit devant m, il est donc moins coûteux d'implémenter la liste d'adjacence.

3) Représenter graphiquement un graphe orienté *biparti* G associé au jeu de Nim $(n = 5, p = 2)$. Chaque sommet est de la forme $s = (i,j)$, où $0 \leq i \leq n$ représente le nombre d'objets, et $j = 1$ ou 2 le joueur. Implémenter ce graphe en `python` sous forme d'un dictionnaire `G`, où chaque clé est un sommet `(i,j)` et chaque valeur `G[(i,j)]` représente la liste des successeurs de `(i,j)`.

Solution

On peut représenter le graphe orienté biparti G sous la forme de deux colonnes reliées par des arcs. Chaque colonne correspond à un ensemble S_1 ou S_2 de sommets appartenant aux joueurs $j = 1$ ou $j = 2$.



Solution

On peut implémenter G sous forme d'une liste d'adjacence en utilisant la classe `dict` *(dictionnaire) de* `python`*.*

```
G = {(5,1):[(4,2),(3,2)], (4,2):[(3,1),(2,1)], (3,2):[(2,1),(1,1)], (3,1):[(2,2),(1,2)],
     (2,1):[(1,2),(0,2)], (2,2): [(1,1),(0,1)], (1,1):[(0,2)], (1,2):[(0,1)], (0,2):[],
     (0,1):[]}
```

4) En pratique, il est fastidieux de définir manuellement le graphe biparti associé au jeu de Nim pour les grandes valeurs de n. On propose donc de définir une fonction récursive `nim` qui construit le graphe orienté biparti associé au jeu de Nim $(n, p = 2)$. Commenter et compléter le code ci-dessous. Effectuer l'exécution pas à pas de l'appel `G={};nim(G,5,1)`. Exécuter ce code sur `python` puis vérifier que la variable `G` correspond bien au graphe orienté biparti du jeu de Nim $(n = 5, p = 2)$.

```python
def nim(G,n,j):
    """
    Entrée : G (dict), n (int > 1), j (1 ou 2)
    Sortie : None
    """
    adv = 1+j%2
    if n >= 2:
        G[(n,j)] =
        nim(G,n-1,adv)
        nim(G,n-2,adv)
    elif n == 1:
        G[(n,j)] =
        nim(G,n-1,adv)
    else:
        G[(n,j)] =
```

Solution

```python
def nim(G,n,j):
    """
    Entrée : G (dict), n (int > 1), j (1 ou 2)
    Sortie : None
    """
    adv = 1+j%2                             # adversaire de j
    if n >= 2:                              # si sommet final non successeur de n
        G[(n,j)] = [(n-2,adv),(n-1,adv)]    # ajouter successeurs de n à G
        nim(G,n-1,adv)                      # appeler nim sur successeur n-1
        nim(G,n-2,adv)                      # appeler nim sur successeur n-2
    elif n == 1:                            # si sommet final successeur de n
        G[(n,j)] = [(n-1,adv)]              # ajouter successeur de n à G
        nim(G,n-1,adv)                      # appeler nim sur successeur n-1
    else:                                   # si n sommet final (n=0)
        G[(n,j)] = []                       # liste des successeurs vide
```

Solution

On peut effectuer l'exécution pas à pas en suivant à chaque appel de la fonction récursive la valeur des variables locales `n`, `j`, `adv`, `G[(n,j)]`, *ainsi que laquelle des conditions* `n >= 2`, `n == 1` *ou* `n == 0` *est vérifiée. On peut également indiquer le dictionnaire complet* `G` *si la place le permet. La représentation sous forme de tableau est adaptée. Il est nécessaire d'initialiser* `G` *au dictionnaire vide avant le premier appel de la fonction* `nim`.

n	j	adv	condition	G[(n,j)]
5	1	2	n>=2	[(4,2),(3,2)]
4	2	1	n>=2	[(3,1),(2,1)]
3	1	2	n>=2	[(2,2),(1,2)]
2	2	1	n>=2	[(0,1),(1,1)]
1	1	2	n==1	[(0,2)]
0	2	1	n==0	[]
...	...	...	...	...

On représente ci-dessus uniquement la première pile d'exécution, qui mène au sommet final $(0, 2)$. Il faut ensuite remonter la pile d'exécution en complétant les autres appels récursifs. On remarque que cet algorithme n'est pas optimal, puisqu'il repasse plusieurs fois par les mêmes sommets. Nour verrons plus loin comment optimiser le parcours pour l'algorithme min-max.

5) Bonus : généraliser la fonction `nim` à une valeur de p quelconque. Utiliser cette fonction pour générer le graphe représentatif du jeu de nim ($n = 9, p = 3$).

Solution

On peut pour cela essayer d'ajouter itérativement les successeurs possibles de `(n,j)` *en utilisant une boucle* `while`,*tout en appelant récursivement la fonction* `nim` *pour chaque successeur ajouté.*

```python
def nim(G,n,j,p):
    """
    Entrée : G (dict), n (int > 1), j (1 ou 2), p (int < n)
    Sortie : None
    """
    G[(n,j)] = []
    adv = 1+j%2
    i   = 1
    while n >= i and i <= p:
        G[(n,j)].append((n-i,adv))
        nim(G,n-i,adv,p)
        i += 1
```

Nombre de Grundy — On se propose de déterminer les stratégies gagnantes du jeu de Nim à partir d'un théorème mathématique appelé théorème de Sprague-Grundy. À chaque sommet (i, j) appartenant au joueur j contenant i objets, on associe le nombre noté $g(i)$, appelé nombre de Grundy, défini par :

$$\begin{cases} g(i) = i \text{ si } 1 \leq i \leq p \text{ (sommet final)}, \\ g(i) = \min\{\mathbb{N} \backslash \{g(i-\ell), \forall \ell \in [\![1, p]\!]\}\} \text{ sinon.} \end{cases}$$

L'ensemble $\{g(i-\ell), \forall \ell \in [\![1, p]\!]\}$ représente l'ensemble des nombres de Grundy des successeurs de i. On commence par étudier le jeu de Nim pour $p = 2$. Par exemple, depuis la position $i = 4$ on peut atteindre les

positions 3 et 2. Si $g(3) = 0$ et $g(2) = 2$, alors $g(4) = \min\{1, 3, 4, ...\} = 1$.

6) Montrer par récurrence sur i que $g(i)$ est égal au reste de la division euclidienne de i par 3, soit

$$\forall i \in \mathbb{N}^*, g(i) = r, \text{ avec } i = 3q + r, 0 \leq r \leq 2. \tag{1}$$

Solution

- *Hypothèse*. H_i : $g(i) = r$, *avec* $i = 3q + r, 0 \leq r \leq 2$.
- *Initialisation*. *Pour* $i = 1$, *nous avons* $g(1) = 1$ *par définition. Or la division euclidienne de* 1 *par* 3 *donne* $1 = 3 \times 0 + 1$. *Donc* H_1 *est vraie.*
- *Hérédité*. *Supposons que* H_i *est vraie pour tout* $i \leq n$. *Montrons que* H_{n+1} *est vraie. Notons* $n+1 = 3q+r$ *la division euclidienne de* $n+1$ *par* 3, *avec* $0 \leq r \leq 2$ *et* $q \geq 1$. *Les successeurs de* $n+1$ *sont* n *et* $n-1$. *Étudions les trois cas possibles suivant la valeur du reste* r.
  - (a) *Si* $r = 0$, *alors* $n + 1 = 3q$, *donc* $n - 1 = 3(q - 1) + 2$ *et* $n - 2 = 3(q - 1) + 1$. *Les nombres de Grundy des successeurs de* $n+1$ *valent donc* $g(n-1) = 2$ *et* $g(n-2) = 1$. *Ainsi,* $g(n+1) = 0 = r$.
  - (b) *Si* $r = 1$, *alors* $n + 1 = 3q + 1$, *donc* $n - 1 = 3q$ *et* $n - 2 = 3(q - 1) + 2$. *Les nombres de Grundy des successeurs de* $n + 1$ *valent donc* $g(n - 1) = 0$ *et* $g(n - 2) = 2$. *Ainsi,* $g(n + 1) = 1 = r$.
  - (c) *Si* $r = 2$, *alors* $n + 1 = 3q + 2$, *donc* $n - 1 = 3q + 1$ *et* $n - 2 = 3q$. *Les nombres de Grundy des successeurs de* $n + 1$ *valent donc* $g(n - 1) = 1$ *et* $g(n - 2) = 0$. *Ainsi,* $g(n + 1) = 2 = r$.

  On en conclut que H_{n+1} *est vraie.*
- *Conclusion*. *Par récurrence, on a montré que*

$$\boxed{\forall i \in \mathbb{N}^*, g(i) = r, \text{ avec } i = 3q + r, 0 \leq r \leq 2.}$$

On admet que ce résultat se généralise à p quelconque :

$$\forall i \in \mathbb{N}, g(i) = r, \text{ avec } i = (p + 1)q + r, 0 \leq r \leq p. \tag{2}$$

7) Le théorème de Sprague-Grundy, établi dans les années 1930, affirme que tout sommet $(i, 2) \in S_2$ dont le nombre de Grundy vaut 0 est une position gagnante pour le joueur $j = 1$. En déduire une stratégie gagnante f pour le joueur $j = 1$ suivant la valeur de n. Existe-t-elle toujours ?

Solution

Le joueur $j = 1$ *doit atteindre un sommet dont le reste vaut* 0, *i.e. qui est un multiple de* $p+1$. *On peut donc choisir comme stratégie la fonction*

$$\boxed{\begin{array}{llll} f: & S_1 \backslash F & \to & S_2 \\ & (i,1) & \mapsto & (i-r,2) \text{ si } r \geq 1, \\ & (i,1) & \mapsto & (i-1,2) \text{ si } r = 0, \end{array}}$$

où r *désigne le reste de la division euclidienne de* i *par* $p+1$. *La disjonction de cas pour* $r = 0$ *est nécessaire car* $((i,1),(i,2))$ *n'est pas un arc du graphe. Cette stratégie n'est gagnante que si le joueur* $j = 2$ *ne puisse pas atteindre un sommet* $(i, 2)$ *avec* i *multiple de* $p+1$, *ce qui est possible ssi* n *n'est pas un multiple de* $p+1$.

Algorithme min-max — On souhaite appliquer l'algorithme min-max au jeu de Nim.

8) Représenter l'arbre de décision associé au graphe orienté *biparti* G du jeu de Nim ($n = 5, p = 2$).

9) Appliquer à la main l'algorithme min-max sur ce graphe. En déduire s'il existe une stratégie gagnante pour le joueur $j = 1$. Proposer une stratégie gagnante.

10) Pour implémenter l'algorithme min-max, nous souhaitons parcourir le graphe. Rappeler le principe des deux types de parcours de graphe (largeur et profondeur) et la structure de données utilisée dans chaque cas (pile ou file). À quel type de parcours correspond le code suivant ?

```python
def parcours(G,s0):
    """
    Entrée : graphe G (dictionnaire), sommet initial s0 = (j,i)
    Sortie : None
    """
    atraiter = [s0]
    while len(atraiter) > 0:
```

```
        s = atraiter.pop()
        for v in G[s]:
            atraiter.append(v)
```

Solution

Parcours en largeur : file. Parcours un profondeur : pile.

11) Pourquoi l'algorithme itératif ci-dessus n'est-t-il pas adapté pour appliquer l'algorithme min-max ? On propose à la place de définir une fonction récursive `minmax`. Commenter et compléter le code ci-dessous. Effectuer l'exécution pas à pas de l'appel `minmax(G,(1,5))` sur le dictionnaire `G` défini en Sec. 1 et comparer aux résultats des questions précédentes.

```
def minmax(G,s):
    """
    Entrée : graphe G (dictionnaire), sommet initial s
    Sortie : heuristique h(s) (entier)
    """
    if G[s] == []:
        if s[0] == 1:
            return ...
        else:
            return ...
    else:
        h = [minmax(G,v) for v in G[s]]
        if s[0] == 1:
            return ...
        else:
            return ...
```

Solution

```
def minmax(G,s):
    """
    Entrée : graphe G (dictionnaire), sommet initial s = (j,i)
    Sortie : heuristique h(s) (entier)
    """
    if G[s] == []:
        if s[0] == 1:
            return -1
        else:
            return +1
    else:
        h = [minmax(G,v) for v in G[s]]
        if s[0] == 1:
            return max(h)
        else:
            return min(h)
```

Attracteur — On souhaite calculer l'attracteur du graphe orienté biparti G du jeu de nim $(n = 5, p = 2)$.

12) Rappeler la définition de l'attracteur Que vaut l'ensemble F_1 des sommets finaux pour je joueur $j = 1$? Appliquer l'algorithme de calcul de l'attracteur à la main et en déduire l'attracteur $\mathscr{A}$.

13) On souhaite implémenter le calcul de l'attracteur sur python. Pour cela, on peut se demander comment déterminer l'ensemble des sommets finaux d'un graphe G pour un joueur j. Préciser les spécifications de la fonction `final` et compléter le code suivant. Determiner sur `python` la liste `F1` des sommets finaux pour le joueur $j = 1$.

```
def final(G,j):
    F = []          # initialisation de F
    for s in G:     # boucle sur les sommets s
        ...         # à compléter
    return F        # renvoit sommets finaux F
```

Solution

```python
def final(G,j):
    F = []
    for s in G:
        if G[s] == [] and s[0] == j:
            F.append(s)
    return F
```

On appelle `F1=final(G,1)`.

14) On donne ci-dessous une fonction définie en `python` pour calculer l'attracteur d'une graphe G défini sous forme de dictionnaire. On suppose avoir prédéfini les fonctions `existe` et `pourtout` telles que `existe(G,F,j)`$= \{s \in S_j \backslash F, \exists \text{ arc } (s,t) \text{ avec } t \in F\}$ et `pourtout(G,F,j)`$= \{s \in S_j \backslash F, \forall \text{ arc } (s,t), t \in F\}$, pour tout joueur $j = 1, 2$. Effectuer une exécution pas à pas de la commande `attract(G,F1)` où G désigne le graphe du jeu pour $n = 5$ et $p = 2$. Que vaut l'attracteur $\mathscr{A}$? Comparer aux stratégies gagnantes obtenues par les autres méthodes.

```python
def attract(G,F):
    """
    Entrée : graphe du jeu G (dict), sommets finaux F (list), joueur J (int)
    Sortie : attracteur A (list)
    """
    E = existe(G,F,1)   # de type list
    P = pourtout(G,F,2) # de type list
    A = F + E + P
    if E == [] and P == []:
        return A
    else:
        return attract(G,A)
```

Solution

Le sommet gagnant pour J_1 (qui appartient à J_2) est $(2,0)$. Donc $F = \{(2,0)\}$.
Exécution pas à pas de `attract(G,F)`. *<u>Erratum</u> : pour que la condition du* `if` *soit correcte, il faudrait préciser que* `existe` *et* `pourtout` *renvoie uniquement les éléments qui ne sont pas déjà compris dans F. On va supposer que c'est le cas par la suite.*

- $E =$ `existe(G,F,1)` $= [(1,1),(1,2)]$ *puisqu'on peut atteindre* 0 *à partir de* 1 *et* 2.
- $P =$ `pourtout(G,F,2)` $= []$ *est vide puisque* $(2,0) \in J_2$.
- $A = [(2,0),(1,1),(1,2)]$.
- $E \neq []$ *donc on renvoie* `attract(G,[(2,0),(1,1),(1,2)])`.
  - *Ici* $E = []$, *tandis que* $P = [(2,3)]$.
  - $A = [(2,0),(1,1),(1,2),(2,3)]$.
  - $P \neq []$ *donc on renvoie* `attract(G,[(2,0),(1,1),(1,2),(2,3)])`.
    - *Ici* $E = [(1,4),(1,5)]$, *et* $P = []$.
    - $A = [(2,0),(1,1),(1,2),(2,3),(1,4),(1,5)]$
    - $E \neq []$ *donc on renvoie* `attract(G,[(2,0),(1,1),(1,2),(2,3),(1,4),(1,5)])`.
      - *Ici* $E = []$ *et* $P = []$, *donc la condition* `if` *est vérifiée et on retourne* $A = [(2,0),(1,1),(1,2),(2,3),(1,4),(1,5)]$

Une fois toute la chaîne remontée, le programme retourne l'attracteur

$$A = [(2,0),(1,1),(1,2),(2,3),(1,4),(1,5)].$$

Comme l'origine $(1,5) \in A$ appartient à l'attracteur, il existe une stratégie gagnante. Celle-ci, notée $f : S_1 \to S_2$, s'obtient en suivant la liste des sommets ajoutés à A pour construire l'attracteur. Donc $f(1,5) = f(2,3)$, $f(1,1) = (2,0)$ et $f(1,2) = (2,0)$. La valeur de $f(1,4)$ n'est pas importante. Cette stratégie gagnante est cohérente avec celle obtenue par le théorème de Sprague-Grundy.